Problem 7.1 – Fixing Comments

Use Euclid's algorithm to calculate the GCD.

```
private long GCD(long a, long b)
{
// Repeat until remainder is 0
for (;;)
{
// Compute remainder of a divided by b
long remainder = a % b;

   // If remainder is 0, b is the GCD
    if (remainder == 0) return b;

    // Update values for next iteration
    a = b;
    b = remainder;
}


}
```

Problem 7.2 – When Do Bad Comments Happen?

1. When code is modified but comments are not updated
2. When comments describe what the code used to do instead of what it currently does

---

Problem 7.4 – Offensive Programming

Add checks to handle invalid or unexpected inputs:

- Ensure both inputs are not zero
- Handle negative values using absolute value
- Prevent division/modulo by zero

Example:

```
if (a == 0 && b == 0)
throw new IllegalArgumentException("Both inputs cannot be zero");

a = Math.abs(a);
b = Math.abs(b);
```

Problem 7.5 – Should You Add Error Handling?

Yes, but only where necessary.

- Add error handling for invalid inputs such as both values being zero
- Avoid overcomplicating simple and stable logic
- Euclid's algorithm is reliable, so minimal error handling is sufficient

---

Problem 7.7 – Top-Down Driving Instructions

1. Start the car
2. Exit current location
3. Navigate to main road
4. Follow directions to nearest supermarket
5. Enter parking lot
6. Park car

Assumptions:

- Driver knows how to operate the car
- GPS or knowledge of directions is available
- Roads are accessible

Problem 8.1 – Testing isRelativelyPrime()

```
boolean isRelativelyPrime(int a, int b) {
return Math.abs(GCD(a, b)) == 1;
}

void testRelativelyPrime() {
int[][] testCases = {
{8, 9},
{21, 35},
{1, 100},
{0, 1},
{0, 2},
{-3, 7},
{100, 10}
};

for (int[] test : testCases) {
   System.out.println(test[0] + "," + test[1] + " -> " +
      isRelativelyPrime(test[0], test[1]));
}
```

}

Problem 8.3

(a) Techniques used:

- Black-box testing
- Some white-box testing

(b) When to use:

- Black-box: when testing correctness of outputs
- White-box: when verifying internal logic and paths
- Gray-box: when combining both approaches

Problem 8.5 – Implementation and Bugs

- The code is mostly correct
- Possible issues:
    - Language-specific syntax (e.g., Math.abs)
    - Behavior of modulo with negative numbers

Benefits of testing:

- Identifies edge cases (0, negatives, ±1)
- Confirms correctness of logic
- Validates assumptions

Problem 8.9 – Exhaustive Testing

Exhaustive testing is a type of black-box testing because it tests all possible inputs without considering internal implementation.

Problem 8.11 – Lincoln Index

Alice found 5 bugs
Bob found 4 bugs
Overlap between Alice and Bob is 2 bugs

Estimated total bugs:
$N = (5 \times 4) / 2 = 10$

Total unique bugs found = 10

Bugs still at large:
10 − 10 = 0

Problem 8.12 – No Overlap Case

If there is no overlap, the formula becomes undefined (division by zero).

Meaning:

- Testers are not sampling the same bug space
- The estimate is unreliable

Lower bound:

- At least the total number of unique bugs already found